

RAG ChatBot

1. Import Libraries

```
import streamlit as st
import os
from langchain_groq import ChatGroq
from langchain_openai import OpenAIEmbeddings
from langchain_huggingface import HuggingFaceEmbeddings
from langchain.text_splitter import RecursiveCharacterTextSplitter
from langchain.chains.combine_documents import create_stuff_documents_chain
from langchain_core.prompts import ChatPromptTemplate
from langchain.chains import create_retrieval_chain
from langchain_community.vectorstores import FAISS
from langchain_community.document_loaders import PyPDFDirectoryLoader
```

Load Grok API Key:

```
from dotenv import load_dotenv
load_dotenv()

## load the GROQ API Key
groq_api_key=os.getenv("GROQ_API_KEY")
```

2. Define LLM Model

```
llm=ChatGroq(model_name="Llama3-8b-8192",groq_api_key=groq_api_key)
```

3. Define Prompt

```

prompt= ChatPromptTemplate.from_template(
    """
    Answer the questions based on the provided context only.
    Please provide the most accurate response based on the question
    <context>
    {context}
    <context>
    Question:{input}

    """
)

```

4. Define Embeddings

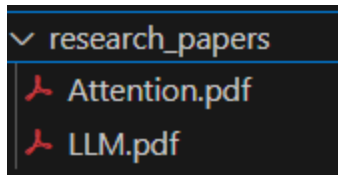
```

def create_vector_embedding():
    if "vectors" not in st.session_state or st.session_state.vectors is None:
        st.session_state.embeddings=HuggingFaceEmbeddings(model_name="a
ll-MiniLM-L6-v2")

        st.session_state.loader=PyPDFDirectoryLoader("research_papers") ## D
ata Ingestion step
        st.session_state.docs=st.session_state.loader.load() ## Document Loadin
g
        st.session_state.text_splitter= RecursiveCharacterTextSplitter(chunk_size
=1000,chunk_overlap=200)
        st.session_state.final_documents=st.session_state.text_splitter.split_docu
ments(st.session_state.docs[:50])
        st.session_state.vectors=FAISS.from_documents(st.session_state.final_d
ocuments,st.session_state.embeddings)

```

research_papers → Folder name



if "vectors" not in st.session_state: :

- This is a crucial check. It ensures that the entire process of creating embeddings and the vector store (which can be time-consuming) only runs **once** during your Streamlit application's lifetime.
- If "vectors" already exists in `st.session_state`, it means the embedding process has already completed, and the function will simply skip all the following steps.

If you didn't have `session_state`, every time your script reran, it would forget everything that happened before.

- **Scenario without `session_state`** : You click a button to load a large file. The script runs, loads the file, and then displays it. But if you then click *another* button, the script reruns from scratch, and it forgets that the file was loaded, so it would have to load it all over again! This would be slow and inefficient.
- **Scenario with `session_state`** : You load the large file, and then you store it in `st.session_state["my_loaded_data"]`. Now, when the script reruns (because you clicked another button), it can check `if "my_loaded_data" in st.session_state:`. If it's there, it knows the data is already loaded and can just use the stored version, saving time and resources.

What is

`st.session_state.docs` ?

- List of documents loaded from PDF files
- So `st.session_state.docs` contains **all** documents extracted from the `"research_papers"` folder.

`st.session_state.docs[:50]` :

- It's important to note that it's only taking the **first 50 documents** loaded. If you have more than 50 research papers, only the initial 50 will be processed for embeddings. You might want to adjust this or remove the slicing (`[:50]`) if you intend to process all documents.

"Take only the first 50 documents from the list"



Important Implication: If you have more than 50 research papers in your `research_papers` directory, only the first 50 will be considered for the Q&A system. The remaining papers will be ignored by this particular `create_vector_embedding` function.



Remaining docs are not split, not embedded, and not stored in FAISS.

? Does `[:50]` mean 50 PDFs?

✗ No, `[:50]` does not mean 50 PDFs.

✓ It means the **first 50 items (documents)** in a Python list — **not 50 files.**

```
if st.button("Document Embedding"):
    create_vector_embedding()
    st.write("Vector Database is ready")
```

`st.button("Document Embedding") :`

- This is a Streamlit command that creates a **button** on your web application's user interface (UI).
- **The text "Document Embedding" will be displayed on the button.**

`if ... :`

: This statement checks if the button has been clicked.

- When a user clicks a button in a Streamlit app, the entire script reruns from top to bottom.
- During that rerun, if that specific button was the one clicked, `st.button("Document Embedding")` will return `True`.
- If the button was *not* clicked (or if the script reran for some other reason, like text input changing), `st.button("Document Embedding")` will return `False`.

5. User Input:

```
st.title("RAG Document Q&A With Groq And Llama3")
```

```
user_prompt=st.text_input("Enter your query from the research paper")
```

6. Answer the Question

```
if user_prompt:
    document_chain=create_stuff_documents_chain(llm,prompt)

    retriever=st.session_state.vectors.as_retriever()

    retrieval_chain=create_retrieval_chain(retriever,document_chain)

    response=retrieval_chain.invoke({'input':user_prompt})
```

```
st.write(response['answer'])
```

if user_prompt:

- This checks if the user has entered any input into a Streamlit text box.
- If `user_prompt` is empty (i.e. the user hasn't typed anything), the rest of the code won't run.

create_stuff_documents_chain : This specific type of chain takes the retrieved documents, "stuffs" them (combines them) into the `context` part of your `prompt`, and then passes the entire formatted prompt to the `llm`

retrieval_chain = create_retrieval_chain(retriever, document_chain)

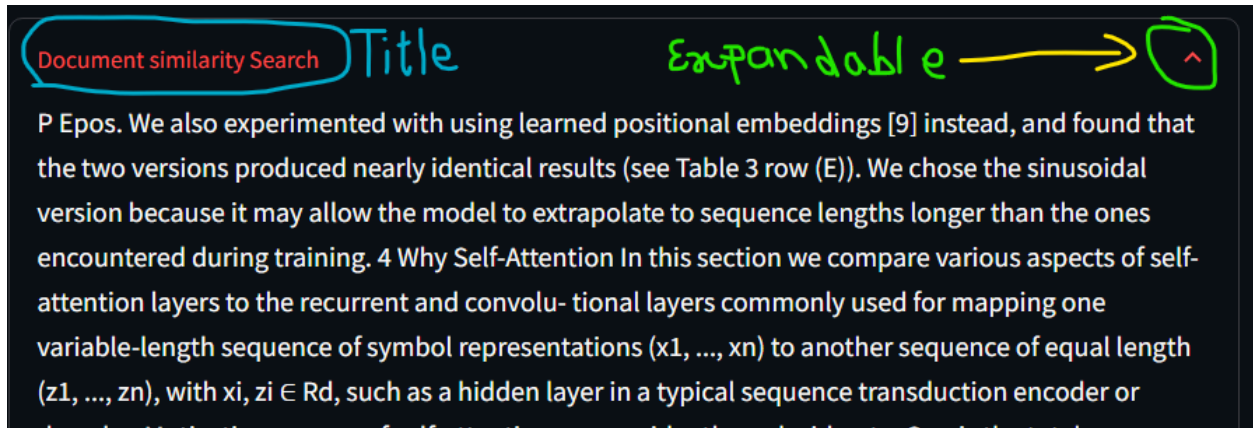
- Combines the `retriever` and the `document_chain` into one `retrieval_chain` (also called a RAG chain).
- This chain does the following:
 1. Takes in user input
 2. Uses retriever to fetch similar documents from the vector store
 3. Feeds the documents + question into the `document_chain` (LLM with the prompt)
 4. Returns the answer

7. Display the source documents (Context)

```
## With a streamlit expander
with st.expander("Document similarity Search"):
    for i, doc in enumerate(response['context']):
        st.write(doc.page_content)
        st.write('-----')
```

What is this code doing?

It displays the **actual text of documents retrieved by the retriever**, i.e. the context used by the model to answer the question — but it shows them inside an **expandable section** in your Streamlit app.



with `st.expander("Document similarity Search")`:

- This creates a collapsible (expandable) section titled "**Document similarity Search**" in the Streamlit interface.
- The user can **click to expand or collapse** this section.
- Helps to **keep the interface clean**, especially if the retrieved documents are long.

◆ `for i, doc in enumerate(response['context']):`

- You're **looping over all the documents** that the retriever returned as context.
- These are stored inside `response['context']`

Why `['context']` ?

Because when you use `create_retrieval_chain()`, the output is a dictionary with keys like:

```
{
  "input": "Your question",
  "context": [List of Documents],
```

```
"answer": "Final Answer"
}
```

- `enumerate(...)` lets you also get the document index `i`, though in this case `i` isn't used directly.

◆ `st.write(doc.page_content)`

- This prints the **main text content** of each document.
- `page_content` is a property of document objects that holds the raw text (not metadata).

Entire Code:

```
import streamlit as st
import os
from langchain_groq import ChatGroq
from langchain_huggingface import HuggingFaceEmbeddings
from langchain.text_splitter import RecursiveCharacterTextSplitter
from langchain.chains.combine_documents import create_stuff_documents_chain
from langchain_core.prompts import ChatPromptTemplate
from langchain.chains import create_retrieval_chain
from langchain_community.vectorstores import FAISS
from langchain_community.document_loaders import PyPDFDirectoryLoader

from dotenv import load_dotenv
load_dotenv()

## load the GROQ API Key
groq_api_key=os.getenv("GROQ_API_KEY")

llm=ChatGroq(model_name="Llama3-8b-8192",groq_api_key=groq_api_key)
```



```

prompt=ChatPromptTemplate.from_template(
    """
    Answer the questions based on the provided context only.
    Please provide the most accurate response based on the question
    <context>
    {context}
    <context>
    Question:{input}

    """

)

def create_vector_embedding():
    if "vectors" not in st.session_state or st.session_state.vectors is None:
        st.session_state.embeddings=HuggingFaceEmbeddings(model_name="a
ll-MiniLM-L6-v2")
        st.session_state.loader=PyPDFDirectoryLoader("research_papers") ## D
ata Ingestion step
        st.session_state.docs=st.session_state.loader.load() ## Document Loadin
g
        st.session_state.text_splitter=RecursiveCharacterTextSplitter(chunk_size
=1000,chunk_overlap=200)
        st.session_state.final_documents=st.session_state.text_splitter.split_docu
ments(st.session_state.docs[:50])
        st.session_state.vectors=FAISS.from_documents(st.session_state.final_d
ocuments,st.session_state.embeddings)

st.title("RAG Document Q&A With Groq And Llama3")

user_prompt=st.text_input("Enter your query from the research paper")

```

```

if st.button("Document Embedding"):
    create_vector_embedding()
    st.write("Vector Database is ready")

if user_prompt:
    document_chain=create_stuff_documents_chain(llm,prompt)
    retriever=st.session_state.vectors.as_retriever()
    retrieval_chain=create_retrieval_chain(retriever,document_chain)

    response=retrieval_chain.invoke({'input':user_prompt})

    st.write(response['answer'])

## With a streamlit expander
with st.expander("Document similarity Search"):
    for i,doc in enumerate(response['context']):
        st.write(doc.page_content)
        st.write('-----')

```

RAG Document Q&A With Groq And Llama3

Enter your query from the research paper

explain THE FREE SPIRIT|

Document Embedding

Based on the provided context, the concept of "The Free Spirit" is described as:

- A group of individuals who are the forerunners and heralds of a new era, beyond traditional morality and societal norms.
- Those who seek to break free from the constraints of conventional thinking and desires, and who are willing to challenge established values and beliefs.
- A class of spirits who are not bound by the prejudices and misconceptions of the past, and who are willing to take risks and push the boundaries of human knowledge and understanding.
- Individuals who are characterized by their independence, creativity, and willingness to question authority and tradition.
- People who are not limited by their personal desires and preferences, but who are driven by a higher purpose and a sense of responsibility to shape the future.
- Those who are not afraid to be alone, to have their own solitude, and to think for themselves, rather than conforming to the norms of society.
- A group that is not interested in leveling and equalizing everything, but rather in creating new values and perspectives that can elevate humanity.